

SIMATS ENGINEERING

CO3 - ASSESSMENT TOOL 1

Case Study

COURSE NAME: DEEP LEARNING

COURSE CODE: MLA0408

COURSE OUTCOME COVERED

CO3: Analyze and apply convolutional neural network architectures, including convolutional layers, filters, parameter sharing, regularization, AlexNet and ResNet, to develop solutions for image classification and real-world computer vision applications. (L4)

CO Weightage: 25%

Assessment Tool Weightage: 25%

QUESTIONS:

Total Marks: 25

Case Study 1: CNN for Automated Medical Image Classification

A healthcare organization wants to develop a deep learning system to automatically classify chest X-ray images into **normal** and **disease-affected** categories. The dataset contains thousands of X-ray images with variations in size, brightness and image quality. A CNN model is proposed because it can automatically learn important visual features such as edges, textures, and abnormal regions without manually designing features.

Question:

1. Design and explain a suitable **CNN architecture** for this medical image classification system.
2. Describe the **motivation for using CNNs** and explain the role of **convolutional layers, filters, pooling layers, and fully connected layers** in the proposed architecture.
3. Explain how **parameter sharing** reduces the number of trainable parameters compared with a fully connected network.
4. Discuss suitable **regularization techniques such as dropout, data augmentation and batch normalization** to reduce overfitting.
5. Finally, explain whether **AlexNet or ResNet** would be more appropriate for this application and justify your choice based on feature learning, network depth, computational requirements and classification performance.

Case Study 2: CNN-Based Autonomous Vehicle Object Recognition

An autonomous vehicle company is developing a vision system that must identify **cars, pedestrians, traffic signs, bicycles, and road obstacles** from images captured by cameras mounted on the vehicle. The system must recognize objects under different lighting conditions, viewing angles, distances and road environments. The development team is considering **AlexNet and ResNet** architectures for building the CNN-based recognition system.

Question:

1. Analyze how a **CNN architecture** can be designed for this autonomous vehicle application.
2. Explain the purpose of the **input, convolution, activation, pooling, and fully connected/output layers** and describe how convolutional **filters progressively learn low-level and high-level features**.
3. Explain the importance of **parameter sharing** in reducing computational complexity and improving translation-related feature detection.
4. Discuss the possible problem of **overfitting** and recommend suitable regularization methods.
5. Compare **AlexNet and ResNet** in terms of architecture, depth, parameter efficiency, training difficulty, vanishing-gradient problems and feature-learning capability.
6. Finally, recommend the most suitable architecture for the autonomous vehicle system and justify your decision based on **accuracy, computational cost and real-time application requirements**.

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding ; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

Case Study 1: CNN for Automated Medical Image Classification

1. Suitable CNN architecture for medical image classification

A suitable CNN for classifying chest X-ray images into normal and disease-affected categories can be designed as follows:

Input Layer:

The chest X-ray images can be resized to $224 \times 224 \times 1$ for grayscale images. Pixel values should be normalized to a suitable range such as 0–1.

Convolutional Layers:

Several convolutional layers can be used to extract visual features. The first layers learn simple features such as edges, lines, and textures. Deeper layers learn more complex features such as lung patterns and abnormal regions.

Activation Function:

The ReLU (Rectified Linear Unit) activation function can be applied after convolution. ReLU introduces non-linearity and helps the network learn complex patterns.

Pooling Layers:

Max-pooling layers can reduce the spatial dimensions of feature maps while retaining important features. This decreases computational requirements and provides some resistance to small changes in image position.

Fully Connected Layers:

After feature extraction, the feature maps are flattened and passed through fully connected layers. These layers combine the learned features for classification.

Output Layer:

Since the problem has two classes, a single neuron with a sigmoid activation can be used. The output represents the probability that the X-ray belongs to the disease-affected class.

A possible architecture is:

Input $\rightarrow$ Conv + ReLU $\rightarrow$ Max Pool $\rightarrow$ Conv + ReLU $\rightarrow$ Max Pool $\rightarrow$ Conv + ReLU $\rightarrow$ Max Pool $\rightarrow$ Flatten $\rightarrow$ Fully Connected $\rightarrow$ Dropout $\rightarrow$ Sigmoid Output

This architecture is suitable because CNNs automatically learn useful visual features from X-ray images without requiring manual feature engineering.

2. Motivation for CNNs and role of different layers

CNNs are highly suitable for medical image classification because images contain spatial patterns. A CNN can automatically learn important features such as edges, textures, shapes, and abnormal regions from chest X-rays.

Convolutional layers:

Convolutional layers apply filters to the input image and generate feature maps. Early convolutional layers detect simple features such as edges, while deeper layers detect more complex patterns.

Filters:

Filters are small matrices of learnable weights. They slide across the image and perform convolution operations. Different filters learn to detect different patterns.

Pooling layers:

Pooling reduces the height and width of feature maps. Max pooling selects the maximum value from a small region. It reduces computation and helps retain important features.

Fully connected layers:

Fully connected layers receive the extracted features and use them to perform the final classification.

Therefore, the CNN performs the overall process:

Image → Feature extraction → Feature reduction → Feature combination → Classification

3. Parameter sharing and reduction of trainable parameters

Parameter sharing is an important property of CNNs. Instead of assigning a separate weight to every pixel connection, a CNN uses the same filter weights at different locations of an image.

For example, consider an image of size 224×224 with a 3×3 filter. The same 3×3 filter is moved across the entire image. Therefore, only 9 weights plus one bias are required for that filter, rather than having separate parameters for every image location.

If a fully connected layer connects every pixel directly to neurons, the number of parameters becomes extremely large.

For example, a 224×224 grayscale image contains:

$224 \times 224 = 50,176$ input values

Connecting these to 1,000 neurons would require approximately:

$50,176 \times 1,000 = 50,176,000$ weights

In contrast, a 3×3 convolutional filter requires only 9 weights (plus bias), regardless of where the filter is applied.

Thus, parameter sharing:

- greatly reduces the number of trainable parameters,
- reduces memory and computational requirements,
- helps prevent overfitting, and
- allows the same feature to be detected at different image locations.

4. Regularization techniques to reduce overfitting

Medical datasets may contain limited numbers of labeled images compared with the complexity of deep neural networks. Therefore, overfitting is an important concern.

a) Dropout

Dropout randomly deactivates a percentage of neurons during training. This prevents the network from becoming overly dependent on particular neurons and encourages it to learn more general features.

For example, a dropout rate of 0.5 can be applied before the final fully connected layer.

b) Data augmentation

Data augmentation artificially creates additional training examples by applying transformations to existing X-rays. Suitable transformations include:

- small rotations,
- horizontal translations,
- small scaling,
- cropping,
- brightness/contrast adjustments.

The transformations should be medically appropriate because unrealistic transformations could introduce misleading patterns.

c) Batch normalization

Batch normalization normalizes intermediate activations during training. It can make training more stable and often allows faster convergence. It can also provide a regularization effect and reduce sensitivity to initialization.

Therefore, combining data augmentation, dropout, and batch normalization can significantly reduce overfitting and improve generalization.

5. AlexNet vs ResNet and recommended architecture

For this medical image classification task, ResNet would generally be more appropriate than AlexNet.

Feature	AlexNet	ResNet
Network depth	Relatively shallow	Can be very deep
Feature learning	Good	More powerful hierarchical feature learning
Training deep networks	More difficult as depth increases	Easier because of residual connections
Vanishing-gradient problem	More susceptible	Significantly reduced
Parameter efficiency	Less efficient for modern deep networks	Generally better for deeper architectures
Classification performance	Good baseline	Usually stronger on complex image tasks

ResNet uses residual/skip connections, which allow information and gradients to flow more easily through a deep network. This makes it possible to train substantially deeper networks without suffering as severely from degradation and vanishing-gradient problems.

For chest X-rays, deeper feature extraction can help the model identify complex patterns associated with diseases. Transfer learning using a pretrained ResNet can also reduce training requirements when the medical dataset is not extremely large.

Therefore, ResNet is the preferred choice because it provides stronger feature-learning capability and generally better classification performance while making deep-network training easier. Computational requirements should still be considered, and a smaller ResNet variant can be selected when hardware resources are limited. The assessment specifically asks for the choice to be justified using feature learning, depth, computational requirements, and classification performance.

Case Study 2: CNN-Based Autonomous Vehicle Object Recognition

1. CNN architecture for autonomous vehicle object recognition

An autonomous vehicle needs to identify multiple object categories such as cars, pedestrians, traffic signs, bicycles, and road obstacles under different conditions.

A suitable CNN architecture can be:

Input Image → Convolution → ReLU → Pooling → Convolution → ReLU → Pooling → Deeper Convolution Layers → Flatten/Global Average Pooling → Fully Connected Layer → Output

The input layer receives images from vehicle-mounted cameras.

The convolutional layers extract visual features from the images. Early layers identify basic patterns such as edges and corners, while deeper layers learn object-specific features.

Pooling layers reduce feature-map dimensions and computational cost.

The final layers use the extracted features to classify objects into categories such as:

- Car
- Pedestrian
- Bicycle
- Traffic sign
- Road obstacle

For a simple image classification system, the output layer can use softmax activation to produce probabilities for multiple object classes.

For a real autonomous vehicle, however, object detection rather than simple image classification would normally be required because the system must identify both *what* an object is and *where* it is in the image.

2. Purpose of CNN layers and progressive feature learning

Input layer:

Receives camera images and preprocesses them into a suitable size and numerical representation.

Convolutional layers:

Apply learnable filters to extract visual features. They preserve spatial relationships between pixels.

Activation layer:

ReLU introduces non-linearity, allowing the CNN to learn complex relationships.

Pooling layer:

Reduces the spatial size of feature maps and lowers computational requirements while retaining important information.

Fully connected/output layers:

Combine the extracted features and produce the final classification probabilities.

CNN filters progressively learn different levels of features.

For example:

First layers:

Edges, lines, corners, and simple textures.

Middle layers:

Shapes, curves, wheels, windows, signs, and other object components.

Deep layers:

Complete high-level patterns representing cars, pedestrians, bicycles, traffic signs, and obstacles.

Thus, the CNN transforms raw camera pixels into increasingly meaningful representations.

3. Importance of parameter sharing

Parameter sharing means that the same convolutional filter is used at different locations of an image.

This provides two major advantages.

Reduced computational complexity:

A filter has a small number of learnable parameters and is reused across the entire image. Therefore, the CNN requires far fewer parameters than a fully connected network.

Translation-related feature detection:

If a filter learns to detect a particular feature such as an edge or wheel, it can detect that feature regardless of where it appears in the image.

For example, a pedestrian may appear on the left side of one image and the right side of another. The same filters can identify relevant features in both positions.

Therefore, parameter sharing makes CNNs computationally efficient and improves their ability to recognize features at different spatial locations.

4. Overfitting and regularization methods

Autonomous vehicle datasets can contain many images, but the network can still overfit to particular roads, lighting conditions, camera positions, or environments.

Suitable regularization methods include:

Data augmentation:

Training images can be modified using appropriate transformations such as changes in brightness, contrast, scale, cropping, and small rotations. This helps the model handle different environmental conditions.

Dropout:

Dropout randomly removes neurons during training, forcing the network to learn more robust representations.

Batch normalization:

Batch normalization stabilizes the training process and can provide a regularization effect.

Early stopping:

Training can be stopped when validation performance stops improving, preventing excessive fitting to the training data.

Weight regularization:

L1 or L2 regularization can penalize excessively large weights and encourage a simpler model.

These techniques help the CNN generalize to unseen road environments.

5. AlexNet vs ResNet

Factor	AlexNet	ResNet
Architecture	Traditional CNN architecture	CNN with residual/skip connections
Depth	Relatively shallow	Can be much deeper
Parameter efficiency	Less efficient for modern deep architectures	Better suited to deep feature extraction
Training difficulty	Easier than very deep networks but limited in depth	Skip connections make deep training easier
Vanishing gradients	More susceptible in deeper versions	Skip connections help reduce the problem
Feature learning	Learns useful hierarchical features	Learns more complex and powerful hierarchical features
Performance	Good baseline	Generally higher performance on complex visual tasks

AlexNet was an important early deep CNN architecture, but ResNet introduced residual connections that allow networks to become much deeper while maintaining effective gradient flow.

For autonomous driving, the ability to learn detailed hierarchical features is particularly important because objects can vary in size, distance, orientation, lighting, and appearance.

6. Recommended architecture for autonomous vehicles

ResNet is the more suitable choice for the autonomous vehicle application.

The main reasons are:

1. **Higher accuracy:** ResNet can learn more complex visual representations, which helps distinguish objects under difficult conditions.
2. **Better feature learning:** Deeper residual networks can learn both low-level and high-level features effectively.
3. **Improved training:** Skip connections reduce problems associated with training deep networks, particularly vanishing gradients.
4. **Robustness:** ResNet-based models can generalize better when objects appear at different scales, distances, orientations, and lighting conditions.
5. **Real-time consideration:** A very large ResNet may be computationally expensive. Therefore, a smaller ResNet variant or an optimized/resized model should be selected when real-time processing is required.

Overall, ResNet provides a better balance of accuracy and feature-learning capability, while the specific ResNet depth should be chosen according to the vehicle's available GPU/AI hardware and latency requirements.

Conclusion

CNNs are highly suitable for both medical image classification and autonomous vehicle vision because they automatically learn hierarchical spatial features. Convolution and parameter sharing reduce the number of parameters, while pooling reduces computational complexity. Dropout, data augmentation, batch normalization, and other regularization techniques help prevent overfitting. Between AlexNet and ResNet, ResNet is generally the stronger choice for both applications, especially when higher feature-learning capability and classification accuracy are required.